ECE 2564
Project 2: Touhou

Fall 2025

How do I read this document?

Our goal has been to provide you with as much information and tools as possible to help you succeed with the project. Nevertheless, you might not need to read the entire document at first. We recommend that you take a quick look at it to know the main topics covered. Then, carefully read the Game Description and Code Quality sections, and begin your design. Once you have made some progress with your project, carefully study the Validation section. Whenever you have any questions, you should first try to find your answer in this document. The Design Strategy section contains tips and advice on completing certain parts of the project.

Fundamentals

!!! General Guidelines for any assignment

- Before you start working on your assignment:
    - **Skim through the entire project description and carefully read the essential sections marked with exclamation marks (!!!).**
    - Read the starter code carefully and in its entirety.
    - Run the provided **F25_Proj2_golden.out** on your board. This is an executable that we call the **golden solution**.
- Every time you sit down to work on your assignment:
    - Before you start coding, **go to Piazza, and read the questions/notes related to the assignment you are working on, even if you took a two-hour break.**
    - After you finish coding, push your changes to GitHub, **even if you are going to take a two-hour break!**

Starter Code

To get the starter code, follow the instructions for accepting assignments and importing projects from the CCS/GitHub Integration course website page. Please make sure to use GitHub Desktop or CLI. Instead of using the assignment link in the tutorial, you will need to use the below link:

https://classroom.github.com/a/jW1hRR44

!!! Honor Code Policy

You are allowed to copy code from any previous program given to you as part of the course material (HW, class examples, course website, etc…) **However, copying/inheriting code from outside/external resources constitutes plagiarism.** If you have any questions whether your research or design falls under "fair use", do not hesitate to contact the Professor, a GTA, or a ULA.

!!! Generative AI Policy

You are not allowed to use the code generated from Generative AI models such as ChatGPT, Gemini, Copilot… etc. However, you can use these tools to assist in debugging or enhancing the readability of your code.
Similarly, when writing your report, you may use Generative AI to improve and edit what you write, but you cannot use someone else's writing as the input to the chatbot or ask the chatbot to write the report for you.
If you use Gen-AI either to polish your report or to polish your code, you need to download the entire conversation with Gen-AI as a pdf and attach it to your report as an appendix.

!!! Late Policy

Please refer to the syllabus for the late policy, which is very similar to the homework late policy in the past. The only small difference is that for projects, the report is due 24 hours after the code. Late penalties will be assessed based on the greater submission delay between the report and the code.

Golden Solution

We are providing a golden solution executable. The purpose of this file is for you to observe the behavior of the project in action. The executable is called **F25_Proj2_golden.out** and will be available in the Canvas file tab. Follow the instructions here to flash the microcontroller with the executable file. **If there is any difference between the golden solution and the project description, please follow the project description. Please also bring that issue up on Piazza so that we are aware of it.** (A "bonus" golden solution that contains some features that

are considered a bonus may be provided for you. These bonus features are not required and are simply just examples. **Please read this document carefully** so you know what features are actually required.)

!!! Submission Process

The submission process has two steps:

1) Submitting your code:
    a) Putting your code on the cloud:

        You will have to submit your project to GitHub by the deadline. **It is extremely important that you make sure that you have uploaded all your files, including custom images, if any, to GitHub.**

        For any reason, if you are unable to submit your work to GitHub, zip up your work, and fill out the Help Ticket form which has a place for you to upload it.

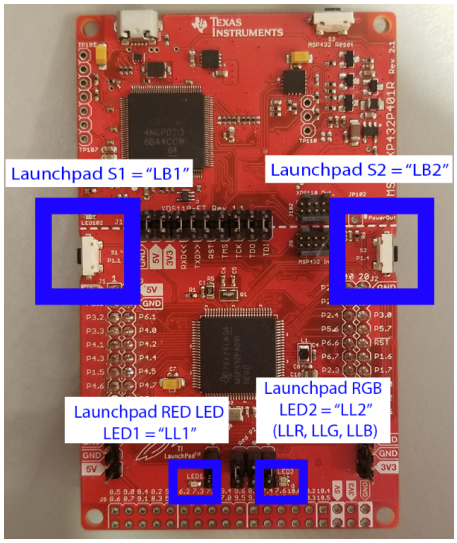
    b) Submitting a screenshot of your Github submission on Canvas.

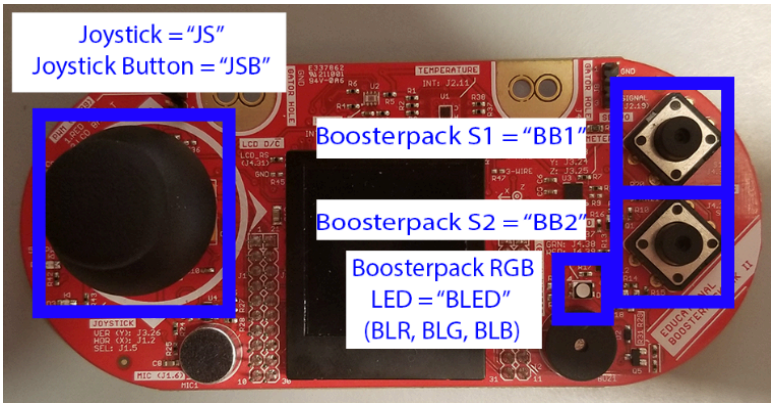
2) Submitting your report, which is due 24 hours after the code deadline.

        You will have to submit your report to Canvas as a pdf.

Button/LED Names

In this project, we will use the LEDs and buttons on both boards. We will use the same naming convention as our previous assignments as shown below. For the BoosterPack LEDs, we will refer to the collection of all three LEDs as BLED, which can be turned into a variety of colors.





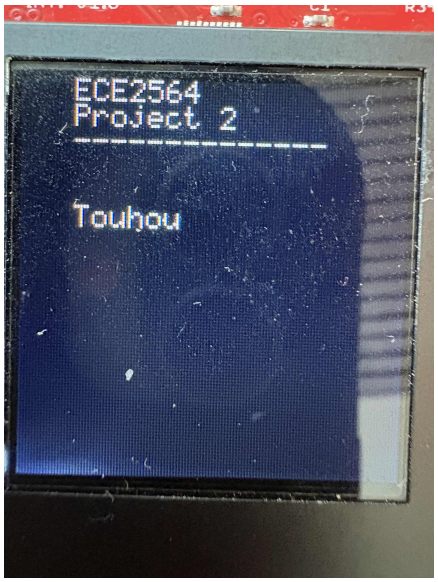
Game Description

Introduction

In this project, you will make a mini version of the japanese game “Touhou”. In this game, you have to dodge bullets sent by an enemy while attacking the enemy back trying to get their health bar down to 0 before yours. The player wins once the enemy’s health reaches 0 and loses if the player’s health reaches 0.

Title Screen

When the player first boots the game, the player should be greeted with a title screen which should display the name of the game and the developer’s name (your name). After **three seconds,** this should switch to the main-menu screen. There should be **no other input responses on this screen. The title screen should have the name of the game and the developers name.**



Main Menu Screen

This screen should provide three (3) options:

- Play the Game
- Go to the Instructions Screen
- Go to the High Scores Screen

Players are given a cursor which **they can move using BB1/BB2:**

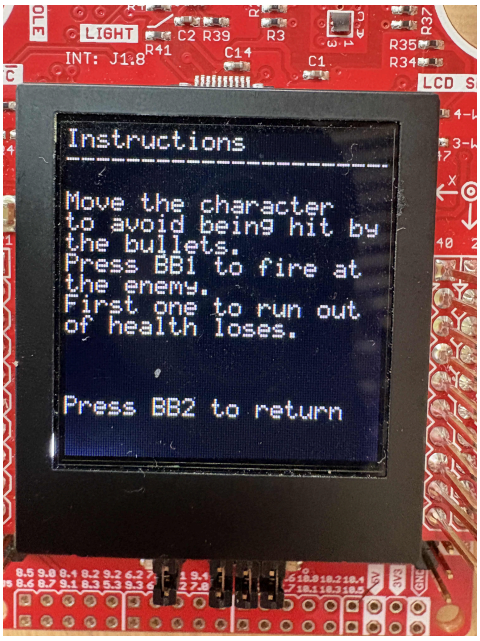
- Pressing BB1 moves the cursor up
- Pressing BB2 moves the cursor down

- The cursor should **circularly increment**, pressing up when the cursor is at the top should move it to the bottom. Pressing down when it's at the bottom should move it to the top

Pressing JSB should go into the selected screen.

Instructions Screen

- This screen should transition from the Main Menu screen.
- It should contain an overall summary, **in your own words.**
- **Pressing BB2 should go back to the Main Menu screen.**



Game Screen

Start of Game

This screen should transition from the Main Menu screen.
When selected, the player is presented with the game environment:

- A player and the player's HP
    - The blue box and circle at the bottom
- The enemy's box and their HP
    - The red box and circle at the top
- Their "score" or time
    - One second corresponds to 100 in-game time units

There should be a counter for the timer on this screen.

Player Controls

- Moving the joystick in any direction causes the player to move
- Pressing BB1 causes a bullet to shoot out
    - If a “raindrop” hits an enemy, the enemy should lose 5 health, likewise for the player

Enemy

- The enemy should randomly move left and right for a random period of time
    - The amount of time and how fast is up to you, but make it reasonable!
- The enemy should cycle between different attack patterns
    - Pattern 1: Vertical “raindrops” starting from the left and making its way to the right side
    - Pattern 2: Horizontal “raindrops” starting from the top and making its way to the bottom
    - Pattern 3: Diagonal “raindrops” that start from one of the top corners and make their way from the top to the bottom
    - **Pattern 4: Dual horizontal “raindrops” where they come from both the left and right hand side and make their way from the top to the bottom of the screen.**
    - **Pattern 5: Random clumps of “raindrops” that randomly spawn from the top of the screen and make their way towards the bottom**
        - Only **one** of the bolded patterns must be implemented

Game Over Screen

- When the game is over (when either HP for the player or enemy goes to 0), the entire game should freeze (all bullets and the enemy stops)
- The screen should either say “You win” or “You lose” depending on the outcome of the game, and prompt you with a score.
    - The score has no official calculation, but it must be based off of the time and player’s remaining health, be creative
    - The score should be 0 if the player loses

High Scores Screen

- This screen should transition from the Main Menu screen.
- It should display the **top five scores in descending order.**
- **Pressing BB2 returns to the Main Menu screen.**

!!! Code Quality

Developing high-quality code is a very important skill for any programmer. A well-written program can be easily tested, updated, and shared with the rest of the team, whereas a low-quality program might “work” for a short period of time but will not be practical in the long run. In this course, you will strive to develop higher quality code to build your skills. We will focus on the below aspects for this purpose.

Comments

You should add enough comments to your code that makes it easy to read and understand. If you are not sure what level of documentation is enough, consult your instructor or your ULA/GTAs. You may also consult with TAs for suggestions on code-comments, but they are NOT the final arbiters of your grade.

No Global Variables

Your code should not use global variables. All interactions between functions should use parameter passing. The only exception may be a custom image (if you implement one) or the hardware timer rollovers.

No Numeric Values

Do not use any hard-coded numeric values inside your code (e.g. `double circumference = 3.15 * 5`) with the exception of 1 and 0. **Instead of hardcoding values in your code, use macros to associate required values with symbolic labels** (e.g. `double circumference = PI * diameter`). This rule does not apply to graphic functions, i.e. values used to change graphics colors or position display elements. However, it is **strongly recommended** to use macros whenever a value for a graphics setting is repeatedly used or if its purpose can be readily ascertained by providing a name (e.g. `#define RED 0x00FF0000` is preferred to using `0x00FF0000` throughout your code).

No Long Functions

All functions, including the main function, should be less than 50 lines of code. This is to encourage you to build modular design with smaller functions that have simple and clear objectives. Comment lines, lines with only curly braces, and empty lines do not contribute to this 50-line count. Points will only be deducted for serious violations of this rule. If a function is very close to the requirement (52, 53 lines, etc.), that will be acceptable. **If a function is above 60 lines of code (with the exception of provided libraries like grlib.h and ImageReformer files) you will receive a 0/30 for this score.** If you find yourself with a function above 60 lines, extract clearly separable portions of code and encapsulate such code into new functions which you may then call in your original over-length function. CCS provides refactoring tools in the editor view which can assist with and automate this process.

Using HAL

You need to have a HAL for all your low-level hardware interactions, including buttons and LEDs. You may use your code from any previous homework or project to implement your HAL. Some code for HAL is given with the project.

Non-Blocking Code

Your entire application must be non-blocking. At any point during the operation of the game, pressing LB1 must turn on LL1. Releasing LB1 must turn off LL1. This operation is to make sure none of your code is blocking the processor from running in an infinite loop responding to user input, similar to a typical bare-metal embedded system.

Bonus Point Opportunities

Before you read about bonus opportunities, **note that bonus features are supposed to build on top of a complete or near-complete project. Please do not use bonus points as a substitute for completing the core functionality of this project.** You want to implement bonus features onto a game which you've gotten to mostly work, perhaps with a couple of bugs that you are still working on. However, if you're unable to get the base functionality of the game working mostly reliably, please implement the rest of your project before asking about bonus features. **ULAs and GTAs will not help you with implementing bonus features until you've already finished the core functionality of this project.**

There are many opportunities to get extra points, the bonus points are capped at **150 bonus points, regardless of how many features you implement.** Along with a standard golden solution, a "bonus" golden solution **may be provided**, which provides some working examples of the annotated features below. You are free to pick some features from the below list, or discuss new features with the instructor for additional bonus opportunities.

- More attack patterns - 2 points each, up to 10 points
- Randomly selected attack pattern - 5 points
- Health bars - 30 points
- Custom images - 25 points
- Player and enemy attacks change color - 25 points
- Power-ups that affect player / enemy - 25 points
- Be creative! Come up with your own.

You are welcome to implement any other extension you are interested in. **However, you need to discuss it with the support staff by making a Piazza post and get confirmation of what you are intending to implement is acceptable and how many points it is worth.**

!!! Bonus Points Grading

The amount of bonus points received is **proportional to the amount of points received from the code-quality section (rounded down)**.

For example,
If you received 150/150 Bonus Points, but only 70/100 Code Quality Points, your resulting bonus points received will be:

```
0.70 (150) → 105 Final Bonus Points
```

Design Strategy

This project will require more effort than previous assignments. It may take several dedicated days, if not the whole two weeks to complete, especially if you struggled in the previous homework assignments or struggle with programming in C in general. Based on our experience in the past, we highly recommend you start projects in this course as soon as possible. **READ THE GAME SPECIFICATION IN ITS ENTIRETY AS SOON AS YOU CAN, so that you can start thinking about design issues you may encounter in your project sooner than later!** Moreover, it is important you **complete the milestone homework assignments** in their entirety, even if the due dates for these assignments have passed, as they are foundational to this project.

In designing Project 2, we recommend the following strategy:

1. Get your peripherals to work (I/O and hardware interfacing).
2. Design your application-level software abstractions on paper (e.g. FSM, pseudocode).
3. Implement each feature delineated in your design outline, one at a time.
4. Use the debugging techniques we learned in earlier assignments: breakpoints, viewing variables, using LEDs, …

As is always the case with any project involving code, always get your inputs and outputs working first before you work on the rest of the code. Import code from those previous examples, and **ensure that your peripherals still behave as expected.**

To test your peripherals, make simple helper functions which validate output as a function of input. The choice for how to do this is ultimately yours, but one simple example includes writing functions to **toggle LaunchPad LEDs when you receive button presses.**

Next, put away your code and get out a pen and some paper. **The best software is designed with a pen and paper, not by hacking away with no stops until it barely works!** Reread the project description and start jotting down some ideas for how you can implement parts of your project. Key components you may wish to consider in your design include the following:

- What kind of data do you need to represent your game? The application struct (and really ANYTHING besides the non-blocking check in the code we have provided) is **completely yours to modify and extend**. When you inevitably run into a problem where you require variables to maintain their values across many calls of your program, the Application struct is a good place to store some of those variables.

- **If you find that the Application struct is getting too large,** consider making other smaller structs to encapsulate variables, like the struct that exclusively holds variables you only need for the actual game.

- How will you draw each UI element onto your LCD? Draw a mock-up on your paper to get a good sense of what the final product should look like, and how each state variable in the game is represented on the LCD.

- **When** do you print an update to the screen? The LCD on your BoosterPack is slow. You cannot afford to clear and redraw everything on the LCD for every pass of your super-loop. You need to be more efficient than that. Try to keep your redraws to the screen to a minimum to reduce lag in your code.

- Think about places in your project where it would make sense to implement an FSM, one of the most powerful organizational tools you have at your disposal. Review the spec and make note of any transitions like button presses or specific inputs that cause a radical shift in the state of your program. Sketch an FSM diagram and make note of all the transitions. Determine which states your program might come back to and which ones will not be revealed.

- What are some ways in which you can incrementally develop your solution? Focus on small objectives, such as printing out the boxes in the right positions, accepting and interpreting the UART input characters and validating them, etc…

- In conjunction with the tip above, try to describe the general tasks that your code performs within specific states, under certain conditions, or on every loop. Breaking down the code into these general tasks can help you develop functions, which keeps your code modular and easy to understand. Give your functions descriptive names that clearly indicate what the functions do. Thinking about the mechanics of the program higher levels first will help you understand the general flow of your code before you start worrying about the low-level details.

- Don't focus too much on making your project look perfect. Get your code working first, then focus on the aesthetics later. Just because your text is perfectly aligned doesn't mean you can ignore the actual game mechanics behind it!

- It's often a good idea to reset variables like arrays to some initial state when necessary. For example, if you only plan on updating part of an array, but the array contains old data that you don't want, set everything in the array to some default state, **then** add in the new values. But if you plan on replacing the **entire array** all at once, there's no sense in doing a reset if everything is going to be overwritten anyways. Whenever you write code, think about that code's purpose and whether it is necessary to reset some variables.

As always, when implementing your code, practice an incremental approach. **Implement each feature you have outlined on your paper one at a time, and immediately test the new feature you have just added so that you can detect possible bugs early!**

Finally when using Git, use your commits wisely. We recommend committing your code after you successfully implement each new feature. This should seem obvious, but don't commit broken code! If you know there is a bug somewhere, fix it before you commit. Each commit you make should be a checkpoint that you can roll your back to if you really must.

Project-Specific Strategies

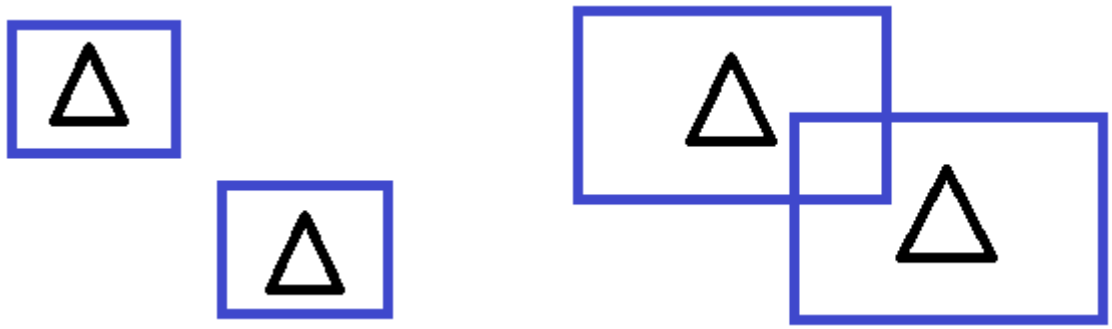
- Be able to verbalize what you want your code to do before you write it. Follow the pattern of "do this thing when this other thing happens".
- Reuse elements. You'll see a lot of "raindrops" on screen, but you write code for more than 1000 (hyperbole) "raindrop" objects it will slow you and the game down.
- Use lots of DEFINEs, it will clean your code and make small edits faster.
- Don't be afraid to experiment. Play around with the code until you like what you wrote.
- Player "raindrops" and enemy "raindrops" behave the same, so do the player and enemy to some extent. Reuse your code.

Collision Detection

To detect when obstacles touch the player (or vice versa), you'll need to essentially solve the **collision detection problem**. Collision detection is a nuanced topic for which there are multiple potential solutions. The two most common techniques people generally use involve either a bounding box strategy or a radial overlap strategy.

In the bounding-box strategy, two objects are considered to overlap when the smallest enclosing rectangle of each object happens to overlap. In the radial overlap strategy, two objects are considered to overlap when the smallest enclosing circle of each object happens to overlap with each other. Examples of each overlap strategy are displayed in figures below:

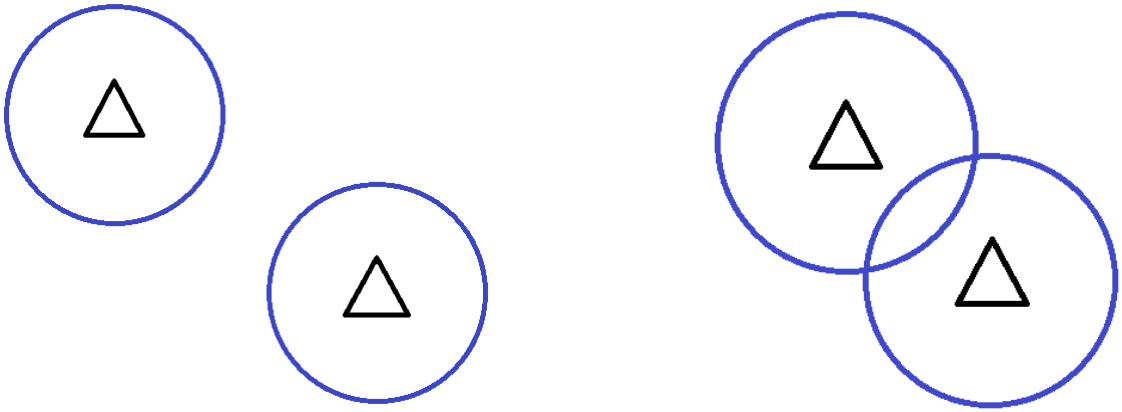
Rectangular Collision



The above image shows an example of rectangular collision detection. Note that for illustration purposes the bounding box has left a lot of extra room around the triangles. In reality, your bounding box should be very close to your object. If you want to implement your collision using

a rectangular approach, you can potentially learn more about it in this link: Determine if two rectangles overlap each other?

Circular Collision



The above image shows an example of radial collision detection. If you want to implement your collision using a radial method, one resource to read is this: How do I detect intersections between a circle and any other circle in the same plane?

Report

Write a short report with the following:

1. **Cover Page:** This should include the title, date, institution (Virginia Tech), and course number. **You must include your name, GitHub username, and a link to your repository on the cover page as well.** You may choose the format, but the cover page needs to be in a professional form.

2. **Report Summary:** In less than 50 words, summarize the purpose and content of your report.

3. **Project Description:** A short section (a paragraph or two) describing what your application can accomplish. Ideally, this matches the Project Description. However, use your own words to describe it. State clearly which parts of the Project Specification are satisfied by your implementation. Take pictures of your game at various stages and include them in your report. If your application cannot perform all the tasks or performs some of them suboptimally, you need to mention them in this section.
   Tip: Use the debugger to help you take screenshots!

4. **Microcontroller-Based Embedded System Architecture:** To the best of your knowledge, draw a high-level architecture of the system you are using for your project.

The drawing should be done on a computer using your program of choice such as Microsoft PowerPoint, Vision, Google Drawings, etc. Your figure should have at least two parts: board-level and chip-level view. In each level, **only** include the peripherals that are involved in this project. You do not need to draw two boards for the BoosterPack and the LaunchPad; a hypothetical unified board will suffice. We have not discussed this in class in detail, but the display is connected over SPI in the MSP432. Write a paragraph or two describing the architecture. Your description should explain what component on the board is connected to what peripheral on the MSP432, and how they communicate to the CPU (Hint: memory-mapped registers).

5. **Code Quality:** Write one or more paragraphs about the various aspects of code quality you had to meet for this project. If your code does not meet the code quality requirements, describe the reason and your challenges. **Create a table or a bulleted list with all the code quality headers in the project description with the number of points you expect to receive for that header. Add an extra row or bullet for the sum of points you expect to get for Code Quality.**

6. **Bonus:** Write one or more paragraphs explaining which bonus features your application achieves (if any), and how you implemented those. **Create a table or bulleted list with the features you have implemented and the number of points you expect to get from those bonus features. Add an extra row or bullet for the sum of points you expect to get for Bonus Points.**

Validation*

Feature	**Point Values and Description**
Title Screen [60]	
Menu Screen [80]	
Instruction Screen [20]	
Options Screen [40]	

Game Screen (Environment) [230]	
Game Screen (Player Interaction and Color) [270]	
Game Over Screen [50]	
High Score Screen [50]	
Code Quality [100]	[10] Code is reasonably commented so that a TA can understand your code. [20] #defines and constants are used instead of hard-coded numbers. [30] No function exceeds 50 lines of code, within reason. Comment lines, lines with only a single curly brace, and empty lines do not contribute to this total. Points will only be deducted for egregious violations of this rule. If a function is very close to the requirement (52, 53 lines, etc) then that is also okay. **If a single function is above 60 lines of code (with the exception of provided libraries like grlib.h and ImageReformer) you will receive a 0/30 for this score.** [20] No global variables are used, with the exception of possibly custom images using the Image Reformer. [20] Your code is completely non-blocking. Pressing LB1 lights up Launchpad LL1 at any time.
Report [100]	[10] No grammatical or spelling errors are present. [10] Cover Page [10] Report Summary [30] Project Description [30] Architecture [10] Code Quality [0] Bonus (only needed if you have completed any bonus features)
Bonus [150]	Report any bonus features and record how many bonus points were earned, provide supporting information on how the bonus should be operated and whether or not it completes the required specifications. If it does not, why justify your design decisions and how many points you deserve.

	In your report, you are required to create a **table or bulleted list** with a row for each bonus feature, and an **extra row** dedicated to the sum of points you **expect** to get for each bonus feature. Points received from bonuses use the following formula: `Final Bonus = [Code Quality %] * [Raw Bonus]`
Total (without Bonus) [1000]	